

Task 1

一、代码结构简要描述

本实验实现了 **32 位单周期 RISC-V CPU**（包含 RV32I 基本指令集及 RV32M 乘除扩展）。模块按功能划分如下：

- **cpu.v**：CPU 顶层模块，**内置 PC**，**向内连接**译码器、寄存器堆、立即数生成模块、ALU、比较器与数据内存控制模块，并**向上级模块开放**连接指令/数据内存的 I/O 端口；负责 **commit** 调试信号输出。
- **decoder.v**：纯组合译码器，根据 **opcode/funct3/funct7** 等生成控制信号（如 **pc_sel**、**rf_we**、**wb_sel**、**alu_op**、**mem_read/mem_write**、**halt** 等）。
- **regfile.v**：32x32 寄存器堆，异步读、时序写，含 debug 只读端口开放给外层 PDU 模块用于随时读取各个寄存器状态。
- **data_mem_ctrl.v**：数据存储器控制，处理非整字访存（字节/半字的对齐、掩码生成、符号/零扩展，以及利用旧数据合并生成整字写入数据）。
- **alu.v**：算术逻辑运算模块。
- **cmp.v**：用于分支跳转的条件判断。
- **imm_gen.v**：根据传入的指令，识别指令类型、生成所需立即数。

二、指令执行的数据通路

指令执行在单个时钟周期内完成，主要步骤包括：

1. **取指**：PC 寄存器的值送入 **imem_raddr**，从指令内存取回 **imem_rdata**（前置判断 PC 是否越界，在 PC 越界时强制重置当前指令为 NOP）。
2. **译码与寄存器读**：**inst** 送入 **decoder** 与 **imm_gen**，**rs1/rs2** 在 **regfile** 中组合读出 **rf_rdata1/2**。
3. **执行**：ALU 的输入由 **alu_src_a/alu_src_b** 选择（**pc/rs1** 与 **imm/rs2**），**alu_op** 选择运算类型，输出 **alu_out**。**cmp** 使用 **rf_rdata1/2** 产生 **cmp_res** 用于判断是否执行分支跳转指令。
4. **访存**：若为 Load/Store，**alu_out** 作为地址送至 **data_mem_ctrl**，顶层将 **dmem_addr** 对齐为整字格式。如果为非整字写，**data_mem_ctrl** 会把新写入的字节与从内存读回的旧数据进行**拼接替换**，最后将拼接好的完整 32 位数据写回内存。
5. **写回与 PC 更新**：**wb_sel** 选择 ALU/MEM/PC+4 写回 **rd**；**next_pc** 由 **pc_sel** 和 **is_jalr** 决定（**JALR** 是 I 型指令，存在 **imm[0]** 位，但实际跳转时仍然需要遵循 PC 对齐的要求，需要将 LSB 清零）。

三、指令译码的逻辑（**decoder.v**）

```
case (opcode)
  // ===== R 型指令 =====
  R_TYPE: begin
    rf_we      = 1;
    wb_sel     = 2'b00;      // ALU 结果
    alu_src_a  = 0;          // rs1
    alu_src_b  = 0;          // rs2

    if (funct7 == 7'b0000001) begin
      // RV32M 扩展指令
      case (funct3)
        3'b000: alu_op = MUL;
```

```

        3'b001: alu_op = MULH;
        3'b010: alu_op = MULHSU;
        3'b011: alu_op = MULHU;
        3'b100: alu_op = DIV;
        3'b101: alu_op = DIVU;
        3'b110: alu_op = REM;
        3'b111: alu_op = REMU;
    endcase
end
else begin
    // RV32I 基本 R 型
    case (funct3)
        3'b000: alu_op = funct7[5] ? SUB : ADD;
        3'b001: alu_op = SLL;
        3'b010: alu_op = SLT;
        3'b011: alu_op = SLTU;
        3'b100: alu_op = XOR;
        3'b101: alu_op = funct7[5] ? SRA : SRL;
        3'b110: alu_op = OR;
        3'b111: alu_op = AND;
    endcase
end
end

// ===== I 型算术逻辑指令 =====
I_TYPE: begin
    rf_we      = 1;
    wb_sel     = 2'b00;
    alu_src_a  = 0;          // rs1
    alu_src_b  = 1;          // imm

    case (funct3)
        3'b000: alu_op = ADD;          // addi
        3'b001: alu_op = SLL;          // slli
        3'b010: alu_op = SLT;          // slti
        3'b011: alu_op = SLTU;         // sltiu
        3'b100: alu_op = XOR;          // xori
        3'b101: alu_op = funct7[5] ? SRA : SRL; // srli / srai
        3'b110: alu_op = OR;           // ori
        3'b111: alu_op = AND;          // andi
    endcase
end

// ===== LOAD 指令 (I 型) =====
LOAD: begin
    rf_we      = 1;
    wb_sel     = 2'b01;          // 从内存读取的数据
    alu_src_a  = 0;
    alu_src_b  = 1;
    alu_op     = ADD;          // 计算地址 rs1 + imm
    mem_read   = 1;
end

// ===== STORE 指令 (S 型) =====

```

```

STORE: begin
    alu_src_a = 0;
    alu_src_b = 1;
    alu_op     = ADD;          // 计算地址 rs1 + imm
    mem_write = 1;
end

// ===== BRANCH 指令 (B 型) =====
BRANCH: begin
    alu_src_a = 1;          // PC
    alu_src_b = 1;          // imm
    alu_op     = ADD;        // 计算跳转目标 PC + imm
    pc_sel     = cmp_res;    // 比较成立则跳转
end

// ===== JAL 指令 (J 型) =====
JAL: begin
    rf_we      = 1;
    wb_sel     = 2'b10;      // PC + 4 (返回地址)
    alu_src_a = 1;          // PC
    alu_src_b = 1;          // imm
    alu_op     = ADD;        // 计算跳转目标 PC + imm
    pc_sel     = 1;          // 无条件跳转
end

// ===== JALR 指令 (I 型) =====
// 跳转目标 = (rs1 + imm) & ~1, 即最低位强制清零
JALR: begin
    rf_we      = 1;
    wb_sel     = 2'b10;      // PC + 4 (返回地址)
    alu_src_a = 0;          // rs1
    alu_src_b = 1;          // imm
    alu_op     = ADD;        // 计算 rs1 + imm
    pc_sel     = 1;          // 无条件跳转
    is_jalr    = 1;          // 标记 JALR, CPU 中将 LSB 清零
end

// ===== LUI 指令 (U 型) =====
LUI: begin
    rf_we      = 1;
    wb_sel     = 2'b00;      // ALU 结果
    alu_src_b = 1;          // imm
    alu_op     = B_OUT;      // 直接输出 imm (imm 已是 {imm[31:12], 12'b0})
end

// ===== AUIPC 指令 (U 型) =====
AUIPC: begin
    rf_we      = 1;
    wb_sel     = 2'b00;
    alu_src_a = 1;          // PC
    alu_src_b = 1;          // imm
    alu_op     = ADD;        // PC + imm
end

```

```
// ===== SYSTEM 指令 =====
SYSTEM: begin
    if (funct3 == 3'b000 && inst20 == 1'b1) begin
        // EBREAK: 停机
        halt = 1;
    end
end

default: begin
    rf_we      = 0;
    mem_write = 0;
    mem_read  = 0;
    halt      = 0;
end
endcase
```

- 通过 **case (opcode)** 将指令分组 (R/I/LOAD/STORE/BRANCH/JAL/JALR/LUI/AUIPC/SYSTEM)。
- 在各分组内根据 **funct3** 选择 **alu_op**、**cmp_op**、**wb_sel**、**rf_we**、**mem_read**、**mem_write** 等信号。
 - RV32M 乘除通过检测 **funct7 == 7'b0000001** 进入扩展表；ADD/SUB、SRL/SRA、SRLI/SRAI 指令通过 **funct7[5]** 进一步区分
- 分支类使用 **cmp_res** 控制 **pc_sel**；JAL/JALR 均设置 **pc_sel=1**，且 JALR 置 **is_jalr=1** 令顶层清除目标地址的 LSB。
- SYSTEM 类在 **funct3==3'b000 && inst20==1'b1** 时识别 EBREAK 并置 **halt=1**。

四、寄存器堆读优先的实现方式 (regfile.v)

```
module regfile (
    input wire      clk,
    input wire      we,           // 写使能
    input wire [4:0] rs1,        // 读寄存器1地址
    input wire [4:0] rs2,        // 读寄存器2地址
    input wire [4:0] rd,         // 写寄存器地址
    input wire [31:0] wdata,      // 写数据
    output wire [31:0] rdata1,    // 读数据1
    output wire [31:0] rdata2,    // 读数据2

    // 调试端口 (供 PDU 读取)
    input wire [4:0] debug_ra,
    output wire [31:0] debug_rd
);

reg [31:0] regs [0:31];

// 同步写 (x0 硬连线为 0, 不允许写入)
always @(posedge clk) begin
    if (we && rd != 5'b0) begin
        regs[rd] <= wdata;
    end
end
end
```

```

// 异步读
assign rdata1 = (rs1 == 5'b0) ? 32'b0 : regs[rs1];
assign rdata2 = (rs2 == 5'b0) ? 32'b0 : regs[rs2];

// 调试读
assign debug_rd = (debug_ra == 5'b0) ? 32'b0 : regs[debug_ra];

endmodule

```

- 同步写：在上升沿用 `regs[rd] <= wdata`（当 `we && rd != 0` 时）提交写入。
- 异步读：`assign rdata1 = (rs1==0) ? 0 : regs[rs1];`（`rdata2` 同理）。
- 寄存器 `x0` 硬件连线赋 0，任何写入 `x0` 的操作都会被忽略，读 `x0` 寄存器始终返回 0。

五、EBREAK 产生 halt 信号并传导到外层 PDU 模块的机制

- 在 `decoder.v` 中，当 `opcode == 7'b1110011`、`funct3 == 3'b000` 且 `inst20 == 1'b1`（EBREAK）时，译码器输出 `halt = 1`。
- 顶层 `cpu.v` 将 `halt` 用于阻止状态更新：`pc_reg` 的更新条件包含 `!halt`，寄存器写使能也被 `!halt` 约束；同时 `commit_halt` 会在调试接口上反映该状态，以便外部调试工具检测停机。

六、非整字访存的实现及非法访存处理（data_mem_ctrl.v）

```

module data_mem_ctrl (
    input wire [31:0] addr,          // 计算出的内存地址（含低2位偏移）
    input wire [2:0] funct3,         // 指令 funct3 字段
    input wire mem_write,           // 写使能
    input wire mem_read,            // 读使能
    input wire [31:0] wdata_in,      // 来自寄存器 rs2 的写数据
    input wire [31:0] rdata_in,      // 从数据存储器读回的 32 位整字
    output reg [31:0] wdata_out,      // 处理后的写数据（字节复制到对应通道）
    output reg [3:0] we_mask,         // 字节写掩码
    output reg [31:0] rdata_out      // 经符号/零扩展后的读数据
);

wire [1:0] offset = addr[1:0]; // 字内字节偏移

// ===== 写操作：生成掩码和对齐数据 =====
reg [31:0] temp_wdata;
always @(*) begin
    we_mask = 4'b0000;
    temp_wdata = wdata_in;

    if (mem_write) begin
        case (funct3)
            3'b000: begin // SB
                // 将最低字节复制到所有 4 个通道，靠 mask 选择实际写入位置
                temp_wdata = {4{wdata_in[7:0]}};
                we_mask = 4'b0001 << offset;
            end
            3'b001: begin // SH

```

```

        // 将低半字复制到两个通道
        temp_wdata = {2{wdata_in[15:0]}};
        // 要求半字对齐, offset[0] 非零则不写
        we_mask = (offset[0]) ? 4'b0000
                  : (offset[1]) ? 4'b1100
                  :                4'b0011;

    end
    3'b010: begin // SW
        temp_wdata = wdata_in;
        we_mask     = 4'b1111;
    end
    default: we_mask = 4'b0000;
endcase
end

// RMW: 直接在控制器内部利用旧数据 (rdata_in) 和写掩模进行合并
wdata_out = (rdata_in & ~{ {8{we_mask[3]}}, {8{we_mask[2]}},
{8{we_mask[1]}}, {8{we_mask[0]}}} })
    | (temp_wdata & { {8{we_mask[3]}}, {8{we_mask[2]}},
{8{we_mask[1]}}, {8{we_mask[0]}}} });
end

// ===== 读操作: 字节选择 + 符号/零扩展 =====
always @(*) begin
    rdata_out = 32'b0;

    if (mem_read) begin
        case (funct3)
            3'b000: begin // LB (有符号字节)
                case (offset)
                    2'b00: rdata_out = {{24{rdata_in[ 7]}}, rdata_in[ 7: 0]};
                    2'b01: rdata_out = {{24{rdata_in[15]}}, rdata_in[15: 8]};
                    2'b10: rdata_out = {{24{rdata_in[23]}}, rdata_in[23:16]};
                    2'b11: rdata_out = {{24{rdata_in[31]}}, rdata_in[31:24]};
                endcase
            end
            3'b100: begin // LBU (无符号字节)
                case (offset)
                    2'b00: rdata_out = {24'b0, rdata_in[ 7: 0]};
                    2'b01: rdata_out = {24'b0, rdata_in[15: 8]};
                    2'b10: rdata_out = {24'b0, rdata_in[23:16]};
                    2'b11: rdata_out = {24'b0, rdata_in[31:24]};
                endcase
            end
            3'b001: begin // LH (有符号半字)
                if (offset[0])
                    rdata_out = 32'b0; // 非对齐, 返回 0
                else if (offset[1])
                    rdata_out = {{16{rdata_in[31]}}, rdata_in[31:16]};
                else
                    rdata_out = {{16{rdata_in[15]}}, rdata_in[15: 0]};
            end
            3'b101: begin // LHU (无符号半字)
                if (offset[0])

```

```

        rdata_out = 32'b0;
    else if (offset[1])
        rdata_out = {16'b0, rdata_in[31:16]};
    else
        rdata_out = {16'b0, rdata_in[15: 0]};
    end
    3'b010: begin    // LW (整字)
        rdata_out = rdata_in;
    end
    default: rdata_out = 32'b0;
endcase
end
end

endmodule

```

1. 存储器拆分与保留旧值写入

- 使用地址低两位 `offset = addr[1:0]` 定位字内字节。
- 对于 Store 操作：
 - **SB**: 把要写入的字节 `wdata_in[7:0]` 铺平为四个通道的数据，并输出对应的 4 位掩码（如 `we_mask = 4'b0001 << offset`）。
 - **SH**: 将 `wdata_in[15:0]` 复制两遍展平为 32 位，根据 `offset[1]` 判断写低半字（`we_mask=4'b0011`）还是高半字（`we_mask=4'b1100`）。
 - **SW**: 要求写入整字，生成全 1 的掩码 `we_mask = 4'b1111`。
- 掩模替换逻辑（旧值合并）：由于处理器外部写接口仅支持 32-bit 整字写入（没有字节使能功能拉出），`data_mem_ctrl.v` 模块直接在内部用由 **dmem** 回传的老数据 `rdata_in` 作为底板，结合生成好的 4 位 `we_mask`，在不需要改变的字节片段上直接截取自内存旧字，再覆盖想修改的新写入部分，最终把经过修改、拼接好的 32 位整字 `wdata_out` 发送给外层做整字写入操作。

2. 读操作的选择与扩展

- **LB/LBU**: 依据 `offset` 提取相应字节并做符号/零扩展。
- **LH/LHU**: 要求半字对齐（`offset[0]==0`），根据 `offset[1]` 选择低半字或高半字并进行符号/零扩展。
- **LW**: 直接返回 `rdata_in`。

3. 非法或跨字访问的处理

- 半字访问在 `data_mem_ctrl.v` 中检测 `offset[0]`：若不对齐，读返回 `32'b0`，写将 `we_mask` 置 `4'b0000`（取消对该字的内部写使能）。
- 顶层模块 `cpu.v` 统一使用抹去了低 2 位后的强制字对齐地址（`dmem_addr = {alu_out[31:2], 2'b00}`）执行访存。

Task 2

使用 `rv32i_test1.asm` 测试程序（运算指令测试）在仿真框架上运行的结果如下：

```
Simulation started...
Diffptest level: FULL.
Hit good trap.
Simulation finished.
Time:                        826
Instruction count:           412
PC:                          0x00400678
Instruction:                  0x00100073
Registers at the end:
[ 0] = 0x00000000 [ 1] = 0x163B9620 [ 2] = 0xDC52630 [ 3] = 0x163B9620
[ 4] = 0x00000000 [ 5] = 0x7A3EC640 [ 6] = 0x00000000 [ 7] = 0x7A3EC640
[ 8] = 0x00000001 [ 9] = 0x00000000 [10] = 0x000005B8 [11] = 0x000005B8
[12] = 0xFFFFFADE [13] = 0x1D5EA000 [14] = 0xDC52630 [15] = 0x00000000
[16] = 0x00000000 [17] = 0xEBB3A000 [18] = 0x00000001 [19] = 0x00000000
[20] = 0x4776D650 [21] = 0x0000007C [22] = 0x4776D650 [23] = 0x00000001
[24] = 0xEBB3A000 [25] = 0x00000000 [26] = 0x0000056D [27] = 0xC8573610
[28] = 0xFFFFFBEF [29] = 0x00000000 [30] = 0x00000001 [31] = 0x000004A2
```

使用 `rv32i_test2.asm` 测试程序（控制与访存指令测试）在仿真框架上运行的结果如下：

```
Simulation started...
Diffptest level: FULL.
Hit good trap.
Simulation finished.
Time:                        884
Instruction count:           441
PC:                          0x0040071C
Instruction:                  0x00100073
Registers at the end:
[ 0] = 0x00000000 [ 1] = 0x100193BF [ 2] = 0x00000000 [ 3] = 0x10029317
[ 4] = 0x00000000 [ 5] = 0x00000000 [ 6] = 0x10029EC8 [ 7] = 0x00000000
[ 8] = 0x1001F720 [ 9] = 0x1003E27A [10] = 0x1001F860 [11] = 0x00000000
[12] = 0x10019699 [13] = 0x00000000 [14] = 0x00000000 [15] = 0x00000000
[16] = 0x00000000 [17] = 0x00000000 [18] = 0x10029AFA [19] = 0x00000000
[20] = 0x1001969A [21] = 0x00000000 [22] = 0x10015F7A [23] = 0x00000000
[24] = 0x00000000 [25] = 0x1003947E [26] = 0x00000000 [27] = 0x00000000
[28] = 0x00000000 [29] = 0x1002A163 [30] = 0x1001959C [31] = 0x00000000
```

Task 3-2

冒泡排序程序

上板运行降序冒泡排序程序（详见 [Labs/Lab2/Task2/task2.asm](#)）前后，查看数据段内存如图所示：

「2026-04-04 17:07:08」 rd 0x10010000 32	「2026-04-04 17:07:53」 rd 0x10010000 32
0x10010000: 0x08646D5A	0x10010000: 0x7C7D7E7F
0x10010004: 0x07336921	0x10010004: 0x78797A7B
0x10010008: 0x010A6031	0x10010008: 0x74757677
0x1001000C: 0x61300F24	0x1001000C: 0x70717273
0x10010010: 0x424B6243	0x10010010: 0x6C6D6E6F
0x10010014: 0x1305457C	0x10010014: 0x68696A6B
0x10010018: 0x4A2F091D	0x10010018: 0x64656667
0x1001001C: 0x52395D3E	0x1001001C: 0x60616263
0x10010020: 0x6F26187E	0x10010020: 0x5C5D5E5F
0x10010024: 0x6E73467B	0x10010024: 0x58595A5B
0x10010028: 0x675F541F	0x10010028: 0x54555657
0x1001002C: 0x5C725E14	0x1001002C: 0x50515253
0x10010030: 0x1C3C510D	0x10010030: 0x4C4D4E4F
0x10010034: 0x207D0240	0x10010034: 0x48494A4B
0x10010038: 0x3A194C56	0x10010038: 0x44454647
0x1001003C: 0x2C712712	0x1001003C: 0x40414243
0x10010040: 0x78321A15	0x10010040: 0x3C3D3E3F
0x10010044: 0x593D1E0C	0x10010044: 0x38393A3B
0x10010048: 0x1B48104D	0x10010048: 0x34353637
0x1001004C: 0x2E7A3549	0x1001004C: 0x30313233
0x10010050: 0x36585774	0x10010050: 0x2C2D2E2F
0x10010054: 0x5B11764F	0x10010054: 0x28292A2B

0x10010058: 0x41533770

0x10010058: 0x24252627

0x1001005C: 0x6563796C

0x1001005C: 0x20212223

0x10010060: 0x346B5547

0x10010060: 0x1C1D1E1F

0x10010064: 0x68297517

0x10010064: 0x18191A1B

0x10010068: 0x28667F0E

0x10010068: 0x14151617

0x1001006C: 0x0B030650

0x1001006C: 0x10111213

0x10010070: 0x6A00772A

0x10010070: 0x0C0D0E0F

0x10010074: 0x252D4404

0x10010074: 0x08090A0B

0x10010078: 0x233F2216

0x10010078: 0x04050607

0x1001007C: 0x4E3B382B

0x1001007C: 0x00010203

在 rars 上运行同样的冒泡排序程序前后，查看数据段内存如图所示：

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x08646d5a	0x07336921	0x010a6031	0x61300f24	0x424b6243	0x1305457c	0x4a2f091d	0x52395d3e
0x10010020	0x6f26187e	0x6e73467b	0x675f541f	0x5e725e14	0x1c3e510d	0x207d0240	0x3a194c56	0x2e712712
0x10010040	0x78321a15	0x593d1e0e	0x1b48104d	0x2e7a3549	0x36585774	0x5b11764f	0x41533770	0x6563796c
0x10010060	0x346b5547	0x68297517	0x28667f0e	0x0b030650	0x6a00772a	0x252d4404	0x233f2216	0x4e3b382b

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x7e7d7e7f	0x78797a7b	0x74757677	0x70717273	0x6e6d6e6f	0x68696a6b	0x64656667	0x60616263
0x10010020	0x5e5d5e5f	0x58595a5b	0x54555657	0x50515253	0x4c4d4e4f	0x48494a4b	0x44454647	0x40414243
0x10010040	0x3e3d3e3f	0x38393a3b	0x34353637	0x30313233	0x2c2d2e2f	0x28292a2b	0x24252627	0x20212223
0x10010060	0x1e1d1e1f	0x18191a1b	0x14151617	0x10111213	0x0c0d0e0f	0x08090a0b	0x04050607	0x00010203

测试程序 (rv32i_test1/2.asm)

上板运行和在 rars 上运行 rv32_test1.asm 后，读取 32 个寄存器结果如图所示：

[2026-04-04 17:37:45] rr
Register 0: 0x00000000

Register 1: 0x163B9620

Register 2: 0xDCf52630

Register 3: 0x163B9620

Register 4: 0x00000000

Register 5: 0x7A3EC640

Register 6: 0x00000000

Register 7: 0x7A3EC640

Register 8: 0x00000001

Register 9: 0x00000000

Register A: 0x000005B8

Register B: 0x000005B8

Register C: 0xFFFFFADE

Register D: 0x1D5EA000

Register E: 0xDCf52630

Register F: 0x00000000

Register 10: 0x00000000

Register 11: 0xEBB3A000

Register 12: 0x00000001

Register 13: 0x00000000

Register 14: 0x4776D650

Register 15: 0x0000007C

Register 16: 0x4776D650

Register 17: 0x00000001

Register 18: 0xEBB3A000

Register 19: 0x00000000

Register 1A: 0x0000056D

Register 1B: 0xC8573610

Register 1C: 0xFFFFFBEF

Register 1D: 0x00000000

Register 1E: 0x00000001

Register 1F: 0x000004A2

Name	Number	Value
zero	0	0x00000000
ra	1	0x163b9620
sp	2	0xdcf52630
gp	3	0x163b9620
tp	4	0x00000000
t0	5	0x7a3ec640
t1	6	0x00000000
t2	7	0x7a3ec640
s0	8	0x00000001
s1	9	0x00000000
a0	10	0x000005b8
a1	11	0x000005b8
a2	12	0xfffffade
a3	13	0x1d5ea000
a4	14	0xdcf52630
a5	15	0x00000000
a6	16	0x00000000
a7	17	0xebb3a000
s2	18	0x00000001
s3	19	0x00000000
s4	20	0x4776d650
s5	21	0x0000007c
s6	22	0x4776d650
s7	23	0x00000001
s8	24	0xebb3a000
s9	25	0x00000000
s10	26	0x0000056d
s11	27	0xc8573610
t3	28	0xfffffbef
t4	29	0x00000000
t5	30	0x00000001
t6	31	0x000004a2

上板运行和在 rars 上运行 rv32_test2.asm 后，读取 32 个寄存器结果如图所示：

[2026-04-04 17:55:49] rr
Register 0: 0x00000000

Register 1: 0x100193BF

Register 2: 0x00000000

Register 3: 0x10029317

Register 4: 0x00000000

Register 5: 0x00000000

Register 6: 0x10029EC8

Register 7: 0x00000000

Register 8: 0x1001F720

Register 9: 0x1003E27A

Register A: 0x1001F860

Register B: 0x00000000

Register C: 0x10019699

Register D: 0x00000000

Register E: 0x00000000

Register F: 0x00000000

Register 10: 0x00000000

Register 11: 0x00000000

Register 12: 0x10029AFA

Register 13: 0x00000000

Register 14: 0x1001969A

Register 15: 0x00000000

Register 16: 0x10015F7A

Register 17: 0x00000000

Register 18: 0x00000000

Register 19: 0x1003947E

Register 1A: 0x00000000

Register 1B: 0x00000000

Register 1C: 0x00000000

Register 1D: 0x1002A163

Register 1E: 0x1001959C

Register 1F: 0x00000000

Name	Number	Value
zero	0	0x00000000
ra	1	0x100193bf
sp	2	0x00000000
gp	3	0x10029317
tp	4	0x00000000
t0	5	0x00000000
t1	6	0x10029ec8
t2	7	0x00000000
s0	8	0x1001f720
s1	9	0x1003e27a
a0	10	0x1001f860
a1	11	0x00000000
a2	12	0x10019699
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x00000000
s2	18	0x10029afa
s3	19	0x00000000
s4	20	0x1001969a
s5	21	0x00000000
s6	22	0x10015f7a
s7	23	0x00000000
s8	24	0x00000000
s9	25	0x1003947e
s10	26	0x00000000
s11	27	0x00000000
t3	28	0x00000000
t4	29	0x1002a163
t5	30	0x1001959c
t6	31	0x00000000